

Generative AI: Music Generation With Transformers

Santiago Villarreal

April 5, 2025

Abstract

This project investigates the use of Transformer-based models for music generation, focusing on symbolic representations of piano performances using MIDI files. By training the model on the MAESTRO dataset and a small collection of Beatles-inspired compositions, the system learns to produce new musical sequences based on patterns found in the original data. The methodology involves converting MIDI files into tokenized sequences, training a custom Transformer architecture, and evaluating the generated music both objectively (through loss metrics) and subjectively (via listening tests). Results indicate that while the model can generate complex note sequences and structure, early outputs often lack harmonic coherence and rhythmic consistency. Extended training on a small dataset led to near-replication of original songs, highlighting risks of overfitting and raising ethical questions regarding originality and copyright.

1 Introduction

1.1 Learning Outcomes

In this project we will:

- Explain the fundamental principles of transformer architectures and their application in sequence modeling tasks, particularly in music generation.
- Design a data preprocessing method that converts MIDI files into model-friendly formats (tokenization).
- Evaluate the quality of generated music using both objective metrics (loss) and subjective listening tests.
- Compare different architectures or hyperparameter settings to assess their impact on the quality and coherence of music generation.

1.2 Problem Statement

The generation of coherent musical sequences using deep learning presents unique challenges in capturing both local melodic patterns and long-term harmonic structures. While Transformers have revolutionized natural language processing through self-attention mechanisms, their adaptation to musical domains remains non-trivial due to music's multi-dimensional nature (rhythm, harmony, dynamics) and temporal dependencies spanning multiple timescales. This project specifically investigates how Transformer architectures can be effectively adapted for symbolic music generation using piano performances from the MAESTRO dataset, while constrained to implementations guided by Foster's framework for creative deep learning. The core challenge lies in developing a model that respects musical theory constraints while maintaining creative flexibility. The scope includes MIDI-based symbolic representation, adaptation of positional encoding strategies for musical time structures, and addressing ethical implications of AI-generated art within the boundaries of existing copyright frameworks.

1.3 Data Source

Data for the completion of the project was taken from the MAESTRO Dataset [?] version 3.0, some additional MIDI files were taken from the Web page.

1.4 Background Theory

In text generation, as discussed in class, the task involves creating coherent and contextually rich information. This challenge is even more pronounced in music generation, where the output must exhibit rhythm, harmony, and structure. Moreover, unlike text generation, music often comprises multiple layers (such as piano, drums, vocals, etc.), which further complicates the task.

Traditional sequence modeling methods such as recurrent neural networks (RNNs) face significant limitations in this domain. RNNs struggle with parallelization [Fos23] and tend to have difficulties maintaining long-term dependencies. These issues are particularly problematic in music generation, where maintaining a consistent melody or theme is essential throughout a piece.

Transformers, introduced in the seminal paper “Attention is All You Need” [VSP⁺17], have addressed many of these limitations. Using attention mechanisms, transformers can encode information, context, and memory into high-dimensional representations. This allows the model to capture long-range dependencies without sacrificing the probabilistic nature of the generation process. In practical terms, this means that a transformer can generate music note-by-note, using a prior sequence of notes to inform the next one. This is achieved through tokenization, where the musical data (i.e., notes) are converted into a sequence of tokens. These tokens are then embedded into a format that the transformer can process effectively, allowing it to learn complex musical patterns and structures.

Furthermore, the ability of transformers to handle multiple layers of information makes them particularly well-suited for music generation tasks. They can simultaneously manage different instrumental or vocal layers, ensuring that the generated composition maintains a cohesive and harmonious structure throughout.

2 Methodology

The code used was followed by the example in KERAS by Joaquin Jimenez [Jim24] with the help of the guide in Foster’s book [Fos23]. First of all, we will be using the Maestro [HSR⁺19] data set which contains MIDI files of piano performances. It uses the MIDI neural processor for MIDI tokenization and kerashub for advanced Keras functionalities. The configuration defines parameters like sequence length, embedding dimensions, batch size, and vocabulary size. In the MIDI library the definitions are as follows..

```
RANGE_NOTE_ON = 128
RANGE_NOTE_OFF = 128
RANGE_VEL = 32
RANGE_TIME_SHIFT = 100
```

This script automates the process of downloading, processing, and training a Transformer model on the Maestro dataset, it has a collection of MIDI files of piano performances. It begins by handling data acquisition, extracting MIDI files with the function `download-maestro`, this function can be change to use other type of data, in the case of the test with data of the battles, I change it with the function `gather-midi-files`. These MIDI sequences are then converted into waveforms, enabling audio playback with the function `visualize-midi`. To prepare the data for model training, the code tokenizes the MIDI sequences into a structured format optimized for deep learning.

```
def encode_midi_task(midi_path):
    import midi_neural_processor.processor as midi_tokenizer
    return midi_tokenizer.encode_midi(midi_path)
```

```
def preprocess_midi_files(file_paths, save_dir=None):
    ...
```

So the code uses the function `preprocess-midi-files` to create the path where the tokens will be saved and uses inside it the function `encode-midi-task` and automatically convert the MIDI information in tokens.

A custom class, `MidiDataset`, is implemented to generate random batches of sequences for the model during training. We define the Transformer model architecture, starting with a custom attention mechanism, `RelativeGlobalAttention` layer, enhances the model’s ability to understand musical patterns by incorporating relative positional encodings. The `DecoderLayer` class takes a

single layer of the Transformer decoder, responsible for applying attention, feed-forward networks, and normalization to refine the representation of the input music sequence. we can stack multiple layers and it will allow to the Transformer model learn complex patterns in music data.

```
class Decoder(layers.Layer):
    ...
    for i in range(self.num_blocks):
        x, w = self.decode_layers[i](x, mask=mask,
                                     training=training)
        weights.append(w)
    ...
```

The Decoder layer will stack depending of the num-blocks a number of decoderlayers. the Music-TransformerDecoder class defines a neural network model that learns to generate music sequences by predicting the next token given a previous sequence of tokens. It utilizes the Transformer architecture and uses techniques like look-ahead masking and top-k sampling to improve the prediction accuracy.

```
def train_loss(y_true, y_pred):
    mask = ops.cast(ops.logical_not(ops.equal(y_true,
        CONFIG.token_pad)), "float32")
    y_true = ops.one_hot(ops.cast(y_true, "int32"),
        CONFIG.vocabulary_size)
    return ops.categorical_crossentropy(y_true, y_pred,
        from_logits=True) * mask
```

For the loss function, the model uses categorical cross-entropy, to ignore padding tokens during loss calculation the code implements the mask, then it converts y-true to one-hot encoding where each token is represented by a vector and with that it calculates the loss assuming that y-pred contains raw output scores instead of probabilities and it's multiply by the mask to exclude the loss contribution from padding tokens. Once trained, the model can generate new music sequences. The code includes functions for creating MIDI files from an initial seed file, with outputs that can be converted into playable audio, allowing for real-time evaluation of the generated music.

2.1 Results and Discussion

After running the original code for 30 epochs, we got a composition that can be heard in wav file "generatedmaestro", first doing a objective analysis the code got a loss of 2.427, we can see that the loss is descending and in the code were we generate the battles music, we can see that the loss went from 5.4 in the first epoch, to 0.07 in the epoch 1000, remember that this was a experiment of overfitting that can be seen in the next section 2.1.1.



Figure 1: Generated with maestro data set after 30 epochs

The model generated all the treble cleff and bass cleff of a piano score, all the notes and even some chords, but, it got messy, it switches the cleffs every time, plays 4-6 notes a time with "one hand", and listening to it is not so satisfactory, I did the subjective test with my wife who had music training in the past and has a good ear for the music (is not needed to know that it is generating nonsense) and got a feedback that it does not have harmony, it lacks of ritm, and it's

just random notes being played. For my perspective, this generation can be improve with more epochs and with human feedback.

2.1.1 Playing with the data

Now that we learn how to generate music, we can change the code to use other types of data, the following experiment is going to be generated with only 23 files from the battles, the files for this experiment were taken from the community in Musescore [zac24][[Thi24](#)][rod24][hia24][224][gug24]. since all the official file had a cost per file, I chose the free ones that the people upload, is necessary to say that these files are not generated by professionals but by amateurs in the area. For this experiment the maximum batch size was of 7.



Figure 2: Generated with the Beatles data set after 30 epochs

Now knowing how the model will generate, I decided to run the code for 1000 epochs, and as we can hear from the attached audio, the code generated a song exactly the same as the songs of the beatles, in this case, it started by a combination of 3-4 songs and then it took the Now and Then song and generated almost the same melody.



Figure 3: Generated with the beattles data set after 1000 epochs (overfitting)

This experiment allowed us to see how the loss covered in 0, and also how the model learns to reproduce the exact information.

2.1.2 Etichs

While working on this project, I faced a major challenge: getting high-quality MIDI files. Hiring musicians to create them from scratch turned out to be expensive, and I quickly realized that official MIDI files have significant value. On top of that, most of the free MIDI files available online come with licenses that don't allow commercial use, which made it difficult to find material I could legally work with.

This brings up important legal and ethical concerns. Using copyrighted music without permission is not only against the law but also unfair to the original artists. It’s important to ensure that any music used in a project is either properly licensed, obtained with permission, or sourced from collections that allow for fair use.

Musicians and composers deserve recognition and compensation for their work. That’s why it’s crucial to cite them properly and, when possible, pay for the rights to use their compositions. This approach not only helps avoid legal problems but also supports the artists who create the music in the first place.

A big part of this project is figuring out how to access MIDI files in a responsible way, whether by collaborating with musicians, purchasing licenses, or using open-source music libraries. Taking these steps ensures that the music generated through this project is both legal and respectful of the artist’s work.

2.2 Summary

This project explores how AI models can learn to create music by learning over existing piano songs. Using a type of model originally designed to understand and generate text (transformers), the project applies this technology to music. The main goal is to teach the computer how to compose music that sounds structured and pleasant to the ear.

The process involved converting musical notes into a tokens, training the model using real piano scores, and then letting it try to create its own music. The project also looked at how well the computer was learning by listening to the results and checking how much it improved over time.

Two main tests were done: one with professional piano recordings and another with Beatles songs. The results showed that the computer could learn patterns in music and start creating its own pieces. However, the quality of the music wasn’t always great, sometimes it sounded messy or lacked rhythm. When trained too long on a small set of songs, the computer ended up just copying those songs instead of creating something new.

Finally, the project highlights an important issue: it’s hard to find music files that are legal to use without paying or asking for permission. This brings up ethical concerns about fairness and respecting the work of real musicians. The project encourages responsible use of music and supporting artists by properly crediting or paying for their work.

References

- [224] Marcus Yen 2. The beatles please please me. *Muscore*, 2024.
- [Fos23] David Foster. *Generative Deep Learning: Teaching Machines To Paint, Write, Compose, and Play*. O’Reilly Media, 2nd edition, 2023.
- [gug24] gugu_{op}00. *Nowandthen~thebeatles(leadsheetwithlyrics)*. *Muscore*, 2024.
- [hia24] hianby123451. Blackbird – the beatles. *Muscore*, 2024.
- [HSR⁺19] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, and Douglas Eck. Enabling factorized piano music modeling and generation with the MAESTRO dataset. In *International Conference on Learning Representations*, 2019.
- [Jim24] Joaquin Jimenez. Music generation with transformer models. *Keras*, 2024.
- [rod24] rodrigo10. Something - the beatles. *Muscore*, 2024.
- [Thi24] ThisJustIn01. The beatles - while my guitar gently weeps (piano accompaniment). *Muscore*, 2024.
- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017.
- [zac24] zackdale56. Here comes the sun – the beatles here comes the sun- piano solo - the beatles. *Muscore*, 2024.